PROJET FYC · COURS EN LIGNE · CRYPTOGRAPHIE APPLIQUÉE À LA FINANCE

Scoring financier sur données chiffrées

Calculer un score de crédit sans jamais voir les données du client : prototype et limites du chiffrement totalement homomorphe (FHE)

Ravin THILAGARASA • **Thomas GRUNDSZTEJN** ESGI Paris · 2026-2027

École Supérieure de Génie Informatique

RÉSUMÉ

Pour calculer un score de crédit, une banque manipule aujourd'hui en clair le revenu, l'endettement et l'historique de paiement de ses clients. Le **chiffrement homomorphe** permet au contraire de calculer **directement sur des données chiffrées** : le serveur produit un score qu'il est incapable de lire. Ce cours part des bases (Paillier, LWE), présente les schémas modernes (BGV/BFV, CKKS, TFHE) et le bootstrapping, puis les applique à un modèle de scoring. Un prototype mesure le prix réel de la confidentialité, comparé aux alternatives (MPC, enclaves, confidentialité différentielle). Le cours est accompagné d'un atelier en ligne de commande, `fhe-cli`, publié sur PyPI.

PROBLÉMATIQUE

Peut-on calculer un score de crédit sur des données qui restent chiffrées de bout en bout, et est-ce exploitable aujourd'hui dans une banque ou une fintech ?

1 Le maillon manquant

Les données sont chiffrées **au repos** (disque) et **en transit** (TLS), mais elles doivent être **déchiffrées pour être traitées**. Dans le cloud, l'hébergeur, un administrateur malveillant ou une fuite mémoire suffisent alors à les exposer, alors que le RGPD (art. 5 et 32) impose minimisation et sécurité du traitement.



Fig. 1. Le FHE protège la dernière étape du cycle de vie de la donnée : le calcul.

2 Calculer sans déchiffrer

Le client garde sa **clé secrète**. Le serveur ne reçoit que des chiffrés et une **clé publique d'évaluation** : il applique le modèle et renvoie un résultat chiffré que seul le client peut ouvrir.

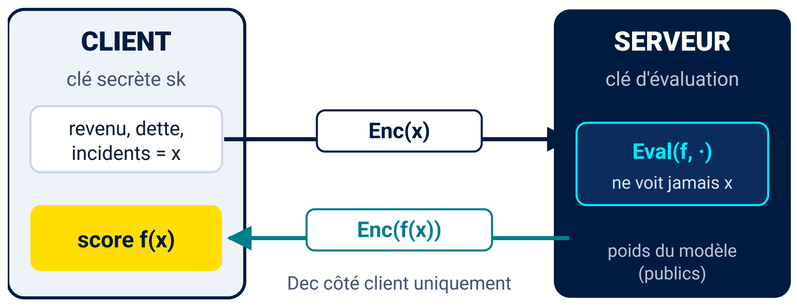


Fig. 2. Protocole mis en œuvre par le client séparé du prototype, face à un serveur honnête mais curieux.

> **Dec**(**Eval**(f, Enc(x_1), …, Enc(x_n))) = f(x_1, …, x_n) (≈ avec CKKS)

3 Le bruit et le bootstrapping

Les schémas modernes reposent sur le problème **LWE / Ring-LWE** (réputé résistant aux ordinateurs quantiques) : chaque chiffré contient un **bruit** aléatoire. Chaque opération l'augmente, la multiplication surtout. Au-delà d'un seuil, le déchiffrement devient faux.

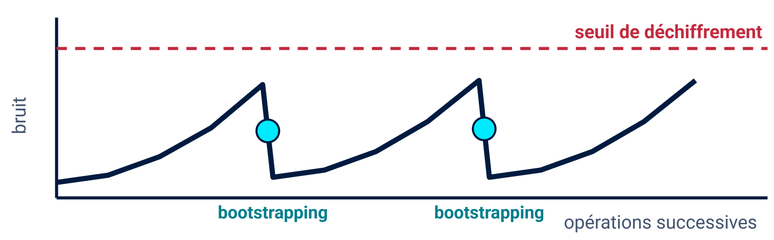


Fig. 3. Le bootstrapping (Gentry, 2009) évalue homomorphiquement le déchiffrement pour « rafraîchir » le chiffré : il lève la limite de profondeur, mais reste l'opération la plus coûteuse. Le prototype n'en a pas besoin.

4 Quatre familles de schémas

Schéma	Opérations	Données	Rôle pour le scoring
Paillier 1999 PHE	+ entre chiffrés, × par un clair	entiers	score linéaire à poids publics
BGV / BFV 2012	+ et ×, nombre limité de niveaux	entiers exacts	règles et comptages exacts
CKKS 2017	+ et × approchés, vecteurs (SIMD)	réels	régression, réseaux de neurones
TFHE 2016-2020	portes logiques, tables de correspondance	bits, petits entiers	comparaisons, seuils de décision

5 Ce que le scoring doit calculer

Le modèle du prototype est un **score linéaire pondéré**, première brique d'une régression logistique :

$$S = 50 + 0{,}005\cdot(R - 2500) - 0{,}9\cdot E - 10\cdot I$$

`R` revenu mensuel net (€) · `E` taux d'endettement (%) · `I` incidents de paiement sur 12 mois. Les **poids** sont publics, les **données** restent chiffrées.

Opération nécessaire	Paillier	CKKS	TFHE
somme de chiffrés	oui	oui	oui
× poids public	oui	oui	oui
× entre deux chiffrés	non	oui	oui
sigmoïde σ(z)	non	approx.	table
décision « S > seuil »	non	coûteux	oui

Une fonction non polynomiale se remplace par un polynôme, par exemple $\sigma(z) \approx 0{,}5 + 0{,}197\,z - 0{,}004\,z^3$ (écart ≤ 0,04 pour $|z| \le 3$). La comparaison au seuil est coûteuse sous chiffrement : le prototype renvoie le score chiffré et laisse le client décider.

6 Prototype : trois chiffrements comparés

Une application web (FastAPI, Docker) **simule** le calcul avec trois briques ; un **client séparé**, seul détenteur des clés, n'envoie au serveur que des chiffrés :

- **AES / Fernet** : sûr, mais aucun calcul n'est possible sur le chiffré.
- **Paillier**, codé à la main : addition chiffrée et produit par les poids publics suffisent au score.
- **CKKS** nivelé (Microsoft SEAL via TenSEAL) : calcul approché sur des réels.

7 Résultats mesurés

Même client (R = 3 200 €, E = 28 %, I = 1, score en clair 18,3) chiffré par le client puis noté par le serveur :

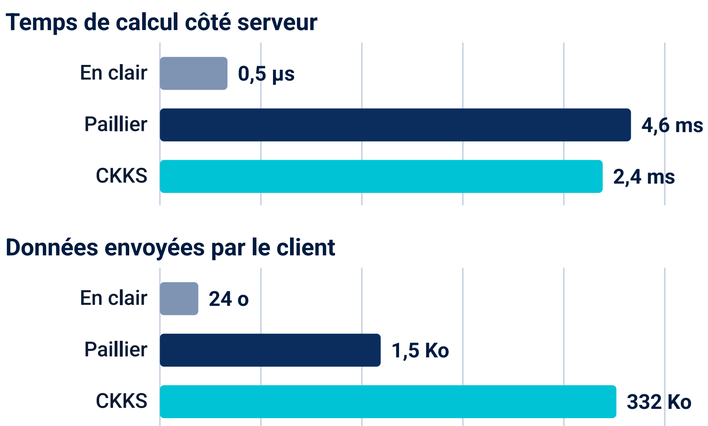


Fig. 4. Médianes de 30 exécutions, échelle logarithmique. Paillier : clé de 2048 bits ; CKKS : N = 8192, échelle 2^{40}. Apple M1, Python 3.13.2, TenSEAL 0.3.18.

380 ms
pour chiffrer côté client avec Paillier (CKKS : 3,4 ms)

×14 000
volume envoyé au serveur (CKKS) par rapport aux données brutes

$1\cdot10^{-5}$
erreur max. CKKS sur 1 000 profils (Paillier : exact après arrondi au millième)

33 Mo
de clés de rotation CKKS, une fois (464 Ko sans rotations)

Côté serveur, les deux schémas répondent en **quelques millisecondes**. Paillier fait payer le **client**, CKKS **la taille** des chiffrés et des clés. Mesures sur une seule machine, sans réseau.

8 Le cours sur Moodle

Un parcours complet de niveau Mastère, du rappel de cryptographie au prototype :

Livre de cours
Paillier pas à pas, LWE, BGV/BFV, CKKS, TFHE, bootstrapping

Leçon interactive
le scoring bancaire sur données chiffrées

État de l'art
travaux académiques, Zama, SEAL, OpenFHE, HElib

Limites et perspectives
coût, normalisation, matériel dédié

Vidéos
cours et correction du cas pratique

Glossaire et examen
72 termes, QCM final de 40 questions

9 Atelier pratique : fhe-cli

Un outil d'exercices en ligne de commande inspiré de rustlings : l'apprenant complète le code dans son éditeur, les tests se relancent à chaque sauvegarde, et la **solution ne se débloque qu'une fois l'exercice réussi**.

```
terminal
$ pip install fhe-cli
$ fhe-cli init && cd fhe-exercices
$ fhe-cli start        # atelier guidé
$ fhe-cli list
  [x] ex8_paillier_homomorphic
  [~] ex9_scoring_paillier
  [ ] ex10_ckks_bonus (bonus)
Progression : 8/10 exercices résolus.
```

1-3 César, XOR, Fernet

4-5 exponentiation, inverse modulaire

6-7 clés et chiffrement Paillier

8-9 homomorphie, score chiffré

10 bonus : calcul avec CKKS

Publié sur PyPI, testé sous Windows, macOS et Linux, avec des exécutables autonomes (Python et TenSEAL embarqués) pour les postes sans Python.

10 Limites et alternatives

Approche	Données en clair ?	Coût	Confiance requise
FHE	non, côté serveur	élevé (taille, calcul)	hypothèse LWE, paramètres, client sain
MPC	non (partagées)	réseau élevé	pas de collusion au-delà d'un seuil
Enclave TEE	dans l'enclave	faible	fabricant du processeur, canaux auxiliaires
Conf. différentielle	oui	faible	protège les statistiques, pas l'entrée

- Le FHE ne prouve pas que le serveur a appliqué **le bon modèle** : l'intégrité du calcul est un autre problème.
- Paramétrage expert (degré, modules, échelle, sécurité 128 bits) et gestion des clés côté client.
- Modèles profonds et décisions non linéaires restent lents sans bootstrapping rapide.

11 Conclusion et perspectives

Pour un **score linéaire**, le calcul sur données chiffrées est **faisable dès aujourd'hui** : quelques millisecondes côté serveur, une erreur négligeable, un serveur qui ne reçoit que des chiffrés. Paillier y suffit. Un usage bancaire demande en plus un modèle validé, l'intégrité du calcul et la gestion des clés.

Ce qui va changer la donne : accélérateurs matériels (GPU, FPGA, ASIC), compilateurs FHE qui masquent les paramètres (Concrete ML, OpenFHE), et normalisation en cours (HomomorphicEncryption.org, ISO/IEC).

RÉFÉRENCES

[1] P. Paillier, *Public-key cryptosystems based on composite degree residuosity classes*, EUROCRYPT 1999.
[2] C. Gentry, *Fully homomorphic encryption using ideal lattices*, STOC 2009.
[3] Z. Brakerski, C. Gentry, V. Vaikuntanathan, *(Leveled) fully homomorphic encryption without bootstrapping*, ITCS 2012.
[4] J. Fan, F. Vercauteren, *Somewhat practical fully homomorphic encryption*, IACR ePrint 2012/144.
[5] J. H. Cheon, A. Kim, M. Kim, Y. Song, *Homomorphic encryption for arithmetic of approximate numbers*, ASIACRYPT 2017.
[6] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène, *TFHE: fast fully homomorphic encryption over the torus*, J. Cryptology 2020.
[7] C. Marcolla et al., *Survey on fully homomorphic encryption, theory, and applications*, IACR ePrint 2022/1602.
[8] A. Al Badawi et al., *OpenFHE: open-source fully homomorphic encryption library*, IACR ePrint 2022/915.
[9] A. Benaissa et al., *TenSEAL: a library for encrypted tensor operations using homomorphic encryption*, 2021 ; Microsoft SEAL.
[10] M. Albrecht et al., *Homomorphic Encryption Standard*, HomomorphicEncryption.org, 2018.

Le cours
`fyc.esgi.fr`

L'atelier
`pip install fhe-cli`